



InfiniFi Protocol

Security Review



Disclaimer

Security Review

InfiniFi Protocol



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

InfiniFi Protocol



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	9
Project Dashboard	13
Risk Section	16
Findings	18
3S-InfiniFi-C01	18
3S-InfiniFi-H01	20
3S-InfiniFi-H02	23
3S-InfiniFi-M01	25
3S-InfiniFi-M02	26
3S-InfiniFi-M03	28
3S-InfiniFi-M04	30
3S-InfiniFi-M05	33
3S-InfiniFi-M06	35
3S-InfiniFi-L01	37
3S-InfiniFi-L02	38
3S-InfiniFi-L03	40
3S-InfiniFi-L04	42
3S-InfiniFi-L05	43
3S-InfiniFi-N01	45
3S-InfiniFi-N02	46
3S-InfiniFi-N03	47

Summary Security Review

InfiniFi Protocol



Summary

Three Sigma audited InfiniFi in a 1.2 person week engagement. The audit was conducted from 9/12/2025 to 11/12/2025.

Protocol Description

InfiniFi is a DeFi protocol that enables users to mint and redeem receipt tokens (iUSD, iETH) against collateral assets (USDC, ETH). The protocol features a sophisticated yield generation system through multiple farm integrations, a locking mechanism for enhanced rewards, and a governance system for farm allocation voting.

Scope Security Review

InfiniFi Protocol

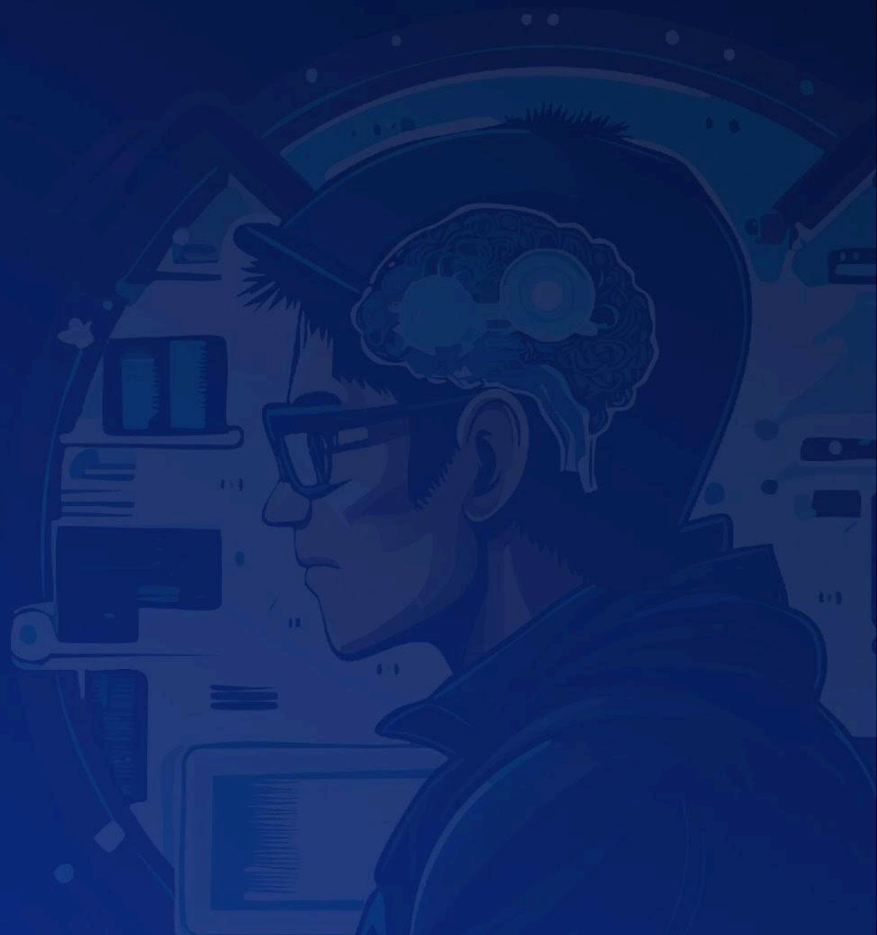


Scope

Filepath	nSLOC
src/integrations/external/ERC7540.sol	141
src/integrations/external/LPTVault.sol	101
src/libraries/GatewayLib.sol	86
src/integrations/farms/AaveV3Farm.sol	82
src/integrations/farms/MapleFarm.sol	75
src/integrations/external/STVault.sol	68
Total	553

Methodology Security Review

InfiniFi Protocol



To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on ImmuneFi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard Security Review

InfiniFi Protocol



Project Dashboard

Application Summary

Name	InfiniFi
Repository	https://github.com/InfiniFi-Labs/infinifi-contracts
Commit	1441778, 66419eb
Language	Solidity
Platform	EVM

Engagement Summary

Timeline	9/12/2025 to 11/12/2025
Nº of Auditors	2
Review Time	1.2 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	1	1	0
High	2	2	0
Medium	6	6	0
Low	5	5	0
None	3	2	1

Category Breakdown

Suggestion	3
Documentation	0
Bug	14
Optimization	0
Good Code Practices	0

Risk Section Security Review

InfiniFi Protocol



Risk Section

- **Unwinding limitation:** Currently the system only supports one redemption per block across all users. The client noted that such limitations will be fixed in v2.

Findings

Security Review

InfiniFi Protocol



Findings

3S-InfiniFi-C01

Incorrect usage of **EnumerableSet.remove** in **LPTVault::_redeem** prevents proper removal of unwinding positions

Id	3S-InfiniFi-C01
Classification	Critical
Impact	Critical
Likelihood	High
Category	Bug
Status	Addressed in #4f02d39 .

Description

The **LPTVault::_redeem** function is responsible for processing redemption requests by claiming matured unwinding positions from the InfiniFi gateway. It manages a per-controller set of unwinding timestamps using OpenZeppelin's **EnumerableSet.UintSet** to track pending redemptions.

The function retrieves the first unwinding timestamp using **_unwindings.at(0)**, which returns the **value** stored at index 0. However, it then attempts to remove this entry by calling **_unwindings.remove(0)**.

OpenZeppelin's **EnumerableSet.remove(uint256 value)** removes elements **by value**, not by index. This means the current implementation attempts to remove the literal value **0** from the set rather than removing the actual **unwindingTimestamp**.

Impact: Since the unwinding position entry remains in the set after redemption, users cannot redeem subsequent positions because the same stale timestamp will be returned on the next **_redeem** call. This causes a revert since the position was already withdrawn from the gateway. The set grows indefinitely with orphaned entries, resulting in a permanent denial of service for affected controllers who have more than one unwinding position.

Recommendation

Replace the incorrect **remove(0)** call with **remove(unwindingTimestamp)** to properly remove the processed unwinding position by its value:

```

function _redeem(uint256, address _controller, address _receiver) internal override
returns (uint256 assetsOut) {
    EnumerableSet.UintSet storage _unwindings = unwindings[_controller];
    require(_unwindings.length() > 0, NoPositions());
    uint256 unwindingTimestamp = _unwindings.at(0);
    (uint256 receiptTokens, bool withdrawable) =
gateway.unwindingPosition(address(this), unwindingTimestamp);
    require(withdrawable, RedemptionNotClaimable(unwindingTimestamp));
-   _unwindings.remove(0);
+   _unwindings.remove(unwindingTimestamp);
    gateway.withdraw(unwindingTimestamp);
    uint256 minAssetsOut = gateway.receiptToAsset(receiptTokens);
    IERC20(gateway.receiptToken()).forceApprove(address(gateway),
receiptTokens);
    return gateway.redeem(_receiver, receiptTokens, minAssetsOut);
}

```

3S-InfiniFi-H01

Missing minimum shares validation in **LPTVault::_requestRedeem** enables denial of service through dust redemption requests

Id	3S-InfiniFi-H01
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #6d508e9 .

Description

The **LPTVault** contract implements the ERC-7540 asynchronous redemption pattern, allowing users to request redemptions that go through an unwinding period before assets can be claimed. When a user calls **requestRedeem**, the internal **_requestRedeem** function initiates the unwinding process via **gateway.startUnwinding**.

The unwinding position in **UnwindingModule** is uniquely identified by both the owner address (the **LPTVault** contract) and **block.timestamp**. This design prevents multiple unwinding positions from being created in the same block, as enforced by the check in **UnwindingModule::startUnwinding**:

```
require(positions[id].fromEpoch == 0, UserUnwindingInProgress());
```

This restriction exists because if multiple users could start unwinding in the same epoch, one user could potentially claim all other users' redemptions since positions share the same identifier.

However, there are two contributing factors to this vulnerability:

1. The unwinding position ID derivation in **UnwindingModule::_unwindingId** only considers the vault address and **block.timestamp**, but does not account for the actual end-user who initiated the redemption request. This means all redemption requests from different users through the same vault in the same block share the same position ID.
2. The **LPTVault::_requestRedeem** function lacks a minimum share amount validation, allowing an attacker to submit redemption requests with negligible (dust) amounts.

Since only one redemption request can succeed per block for the entire vault, an attacker can front-run legitimate redemption requests, causing them to revert with

UserUnwindingInProgress.

Steps to Reproduce:

1. Attacker monitors the mempool for pending **requestRedeem** transactions targeting the **LPTVault** contract
2. When a legitimate user submits a **requestRedeem** transaction, the attacker front-runs it by submitting their own **requestRedeem** with a minimal share amount (e.g., 1 wei worth of shares)
3. The attacker's transaction executes first, calling **UnwindingModule::startUnwinding** and creating a position with ID **keccak256(abi.encode(LPTVault, block.timestamp))**
4. The legitimate user's transaction attempts to execute but reverts because **positions[id].fromEpoch != 0**
5. The attacker can repeat this attack every block at minimal cost, effectively blocking all redemption requests

Recommendation

Option 1: Add a minimum shares requirement to the **_requestRedeem** function to make the attack economically infeasible:

```
+ error MinimumSharesRequired(uint256 minimum, uint256 provided);
+
+ uint256 public constant MIN_REDEEM_SHARES = 1e18; // adjust based on token
decimals and economic considerations
function _requestRedeem(uint256 _shares, address _controller, address _owner)
    internal
    override
    returns (uint256 requestId)
{
+   require(_shares >= MIN_REDEEM_SHARES,
MinimumSharesRequired(MIN_REDEEM_SHARES, _shares));
    unwindings[_controller].add(block.timestamp);
    IERC20(share).safeTransferFrom(_owner, address(this), _shares);
    IERC20(share).forceApprove(address(gateway), _shares);
    gateway.startUnwinding(_shares, uint32(unwindingEpochs));
    return 0;
}
```

Option 2: Modify the **UnwindingModule::startUnwinding** function to derive the position ID from the vault address, **block.timestamp**, and an additional parameter representing the actual end-user who initiated the redemption. This would require updating the **_unwindingId** function to accept a third parameter for the originating user address, and updating the **LPTVault** to pass the **_controller** or **_owner** address through to **gateway.startUnwinding**. This approach addresses the root cause by ensuring each user's unwinding position is uniquely identifiable, allowing multiple users to request redemptions through the same vault within the same block without ID collisions. Special care should be taken to make sure there are no side effects in the out-of-scope contracts.

3S-InfiniFi-H02

Missing operator authorization in ERC7540 requestRedeem allows theft of approved vault shares

Id	3S-InfiniFi-H02
Classification	High
Impact	High
Likelihood	High
Category	Bug
Status	Addressed in #ae92249 .

Description

The **ERC7540::requestDeposit** function is designed to process deposits where a **controller** can act on behalf of an **owner**, and it explicitly enforces that when **_controller** differs from **_owner**, the **_controller** must be a valid operator via **isOperator**, preventing unauthorized use of the **owner**'s asset allowances. In contrast, **ERC7540::requestRedeem** processes share redemptions from the vault but does not perform the same operator check when **_controller** is not equal to **_owner**, even though it can consume share approvals granted to the vault. As a result, any entity that can trigger **ERC7540::requestRedeem** with access to approved shares can redeem on behalf of an **owner** without being a registered operator, enabling theft of shares (and thus underlying assets) that were only intended to be used by authorized controllers.

Recommendation

Align the authorization logic of **ERC7540::requestRedeem** with **ERC7540::requestDeposit** by enforcing the same operator check whenever **_controller** is not equal to **_owner**, so that only valid operators can redeem using an **owner**'s approved shares. A straightforward fix is to add the operator check at the start of **ERC7540::requestRedeem**:

```
- function requestRedeem(address _owner, address _controller, uint256 shares, ...)
external {
+ function requestRedeem(address _owner, address _controller, uint256 shares, ...)
external {
+   if (_controller != _owner) {
+       require(isOperator(_owner, _controller), NotOperator(_owner, _controller));
```

```
+ }
  // existing redemption logic
}
```


3S-InfiniFi-M01

Rounding-up in GatewayLib preview conversion functions leads to inflated quotes and potential redeem denial of service

Id	3S-InfiniFi-M01
Classification	Medium
Impact	Medium
Likelihood	Medium
Category	Bug
Status	Addressed in #0e512aa .

Description

The **GatewayLib::stakedToReceipt**, **GatewayLib::receiptToLocked**, and **GatewayLib::lockedToReceipt** functions provide view/preview conversions between staked assets, receipt tokens, and locked positions, and are expected to be used as conservative estimates for user interfaces and for setting slippage bounds. These functions currently round in favor of the user (rounding up), causing previews such as **stakedToReceipt** to return slightly higher values than what can be realized in actual state-changing conversions. When external logic (e.g., **STVault::_redeem**) derives **minAssetsOut** via **convertToAssets(_shares)** or equivalent preview logic based on these functions, the on-chain redemption can revert because the minimum expected assets are higher than what the vault can actually deliver, effectively enabling a denial of service on redemptions under certain conditions.

Recommendation

Update **GatewayLib::stakedToReceipt**, **GatewayLib::receiptToLocked**, and **GatewayLib::lockedToReceipt** so that all preview conversions use floor-style arithmetic (rounding down) that never overestimates user returns, and ensure that **STVault::_redeem** (and any other functions relying on previews for **minAssetsOut**) are wired to these floor-based conversions. Replace any patterns that emulate ceiling division (for example, adding the denominator minus one before division) with straight integer division, and align all preview math with the exact formulas used in the actual state-changing conversion functions so that **convertToAssets** and similar previews always return values less than or equal to the realizable amounts.

3S-InfiniFi-M02

Incorrect handling of on-contract balances in **AaveV3Farm::liquidity** leads to understated available liquidity

Id	3S-InfiniFi-M02
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #9fa1088 .

Description

In **AaveV3Farm**, the **liquidity** function is intended to report how much **assetToken** can be withdrawn from the strategy at the current time, combining idle **assetToken** held directly by the farm contract with the liquidity available on Aave v3, while handling the case where the Aave pool is paused. The current implementation only caps **assets()** by the **assetToken** liquidity at the **aToken** address and returns zero when Aave is paused, without adding the **assetToken** balance held by **AaveV3Farm** itself, despite the NatSpec indicating that this local balance should be included. Consequently, when Aave is paused, **AaveV3Farm::liquidity** incorrectly reports zero even if the farm still holds withdrawable **assetToken**, and when not paused it underreports liquidity by omitting the on-contract balance, which can cause integrators relying on **liquidity** to mis-estimate available withdrawals or enforce unnecessarily strict limits.

Recommendation

Update **AaveV3Farm::liquidity** to explicitly account for the farm's on-contract **assetToken** balance, return that balance when Aave is paused, and add it to the capped Aave liquidity when the pool is active, aligning behavior with the NatSpec comment and the actual withdrawable funds:

```
function liquidity() public view override returns (uint256) {
    uint256 totalAssets = assets();
+   uint256 availableBalance = ERC20(assetToken).balanceOf(address(this));

    // if aave is paused, cannot withdraw from aave
    address dataProvider =
```

```

IAddressProvider(IAaveV3Pool(lendingPool).ADDRESSES_PROVIDER()).getPoolData
Provider();
    bool isAavePaused = IAaveDataProvider(dataProvider).getPaused(assetToken);
-   if (isAavePaused) return 0;
+   if (isAavePaused) return availableBalance;

    // find the amount of assetToken held by the aToken contract
    // this is the liquidity available on aave for the assetToken
    uint256 availableLiquidity = ERC20(assetToken).balanceOf(aToken);

    // if there is less liquidity on aave than the total assets held by the farm,
    // then the liquidity is the amount of USDC held by the aToken contract that is
available to withdraw
-   return availableLiquidity < totalAssets ? availableLiquidity : totalAssets;
+   return availableBalance + (availableLiquidity < totalAssets ? availableLiquidity :
totalAssets);
    }

```

3S-InfiniFi-M03

Incorrect zero-address validation in **AaveV3Farm::setLendingPool** leads to irreversible contract bricking

Id	3S-InfiniFi-M03
Classification	Medium
Impact	High
Likelihood	Low
Category	Bug
Status	Addressed in #87505f9 .

Description

The **AaveV3Farm::setLendingPool** function allows the governor to update the Aave V3 lending pool address used for deposits and withdrawals. However, the validation logic contains a flaw where it checks the current storage value **lendingPool** instead of the incoming parameter **_lendingPool**.

```
function setLendingPool(address _lendingPool) external
onlyCoreRole(CoreRoles.GOVERNOR) {
    require(lendingPool != address(0), ZeroAddress(_lendingPool));
    lendingPool = _lendingPool;
    emit LendingPoolUpdated(block.timestamp, _lendingPool);
}
```

This allows the governor to set **lendingPool** to **address(0)**, after which the contract becomes permanently bricked. Once **lendingPool** is zero, any subsequent call to **setLendingPool** will revert because the check **lendingPool != address(0)** evaluates to false. The error parameter **ZeroAddress(_lendingPool)** in the require statement further indicates that the developer intended to validate the input parameter, not the storage variable.

When **lendingPool** is set to **address(0)**, all core functionality breaks. The **_deposit** function calls **IAaveV3Pool(lendingPool).supply(...)** which reverts when targeting the zero address. Similarly, **_withdraw** calls **IAaveV3Pool(lendingPool).withdraw(...)** which also reverts. The **liquidity** function attempts to read **IAaveV3Pool(lendingPool).ADDRESSES_PROVIDER()** and fails. This renders the farm

contract completely non-functional, requiring emergency governance action to recover any deposited funds.

Recommendation

Validate the input parameter **_lendingPool** instead of the storage variable **lendingPool**.

```
function setLendingPool(address _lendingPool) external  
onlyCoreRole(CoreRoles.GOVERNOR) {  
- require(lendingPool != address(0), ZeroAddress(_lendingPool));  
+ require(_lendingPool != address(0), ZeroAddress(_lendingPool));  
  lendingPool = _lendingPool;  
  emit LendingPoolUpdated(block.timestamp, _lendingPool);  
}
```

3S-InfiniFi-M04

ERC7540 implementation deviates from EIP-7540 specification in multiple ways

Id	3S-InfiniFi-M04
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #aa1f764 .

Description

The **ERC7540** contract serves as the base abstract contract for vault implementations (**STVault** and **LPTVault**) and is intended to provide EIP-7540 compliant asynchronous deposit and redemption functionality. However, the current implementation contains several deviations from the [EIP-7540 specification](#) that break compatibility with standard tooling and integrations.

Deviation 1: Missing ERC-4626 and ERC-20 interface

EIP-7540 explicitly requires that a compliant vault must be an ERC-4626 vault, which in turn requires full ERC-20 compliance. The **ERC7540** contract uses a separate **share** token to track vault ownership rather than implementing these interfaces directly on the vault contract itself. This means standard ERC-20 functions (**transfer**, **transferFrom**, **approve**) and ERC-4626 methods (**maxDeposit**, **maxWithdraw**, **maxMint**, **maxRedeem**, and **preview*** functions) are not exposed on the vault contract. Generic ERC-7540 and ERC-4626 tooling (routers, SDKs, indexers) that expects these interfaces will revert or behave incorrectly.

Deviation 2: Non-compliant event signatures

The event definitions do not match the EIP-7540 specification. The current events include an extra **timestamp** parameter as the first indexed parameter and use incorrect parameter ordering:

- **OperatorSet(uint256 indexed timestamp, address controller, address operator, bool approved)** should be **OperatorSet(address indexed controller, address indexed operator, bool approved)**

- **DepositRequest**(uint256 indexed timestamp, address controller, address owner, uint256 assets) should be **DepositRequest**(address indexed controller, address indexed owner, uint256 indexed requestId, address sender, uint256 assets)
- **RedeemRequest**(uint256 indexed timestamp, address controller, address owner, uint256 shares) should be **RedeemRequest**(address indexed controller, address indexed owner, uint256 indexed requestId, address sender, uint256 shares)
- **Deposit** event parameters should have **controller** first and **receiver** second

Indexers, SDKs, and UIs that decode logs using the standard IERC7540 ABI will encounter parse errors or silently mis-index data.

Deviation 3: `ERC7540::setOperator` does not support multiple operators

The **setOperator** function stores the operator in a single mapping slot (**operators[msg.sender] = _operator**), which means each controller can only have one active operator at a time. Setting a new operator automatically revokes the previous one. Additionally, when **_approved** is **false**, the function deletes the current operator regardless of the **_operator** parameter passed, but emits an event suggesting the passed **_operator** address was revoked.

Recommendation

For Deviation 1: Implement the full ERC-20 and ERC-4626 interfaces on the vault contract by delegating to the underlying **share** token. Add proper access control to the **share** token to restrict normal users from interacting with this contract, users should interact with the vault instead.

For Deviation 2: Update event signatures to match the EIP-7540 specification:

- event **OperatorSet**(uint256 indexed timestamp, address controller, address operator, bool approved);
- event **DepositRequest**(uint256 indexed timestamp, address controller, address owner, uint256 assets);
- event **RedeemRequest**(uint256 indexed timestamp, address controller, address owner, uint256 shares);
- event **Deposit**(uint256 indexed timestamp, address controller, address receiver, uint256 assets, uint256 shares);
- + event **OperatorSet**(address indexed controller, address indexed operator, bool approved);
- + event **DepositRequest**(address indexed controller, address indexed owner, uint256 indexed requestId, address sender, uint256 assets);
- + event **RedeemRequest**(address indexed controller, address indexed owner, uint256 indexed requestId, address sender, uint256 shares);
- + event **Deposit**(address indexed controller, address indexed receiver, uint256

assets, uint256 shares);

For Deviation 3: Replace the single-operator mapping with a nested mapping to support multiple operators per controller:

```

- mapping(address controller => address operator) public operators;
+ mapping(address controller => mapping(address operator => bool)) public
  operators;
  function setOperator(address _operator, bool _approved) external returns (bool) {
-   if (_approved) {
-       operators[msg.sender] = _operator;
-   } else {
-       delete operators[msg.sender];
-   }
-   emit OperatorSet(block.timestamp, msg.sender, _operator, _approved);
+   operators[msg.sender][_operator] = _approved;
+   emit OperatorSet(msg.sender, _operator, _approved);
    return true;
  }
  function isOperator(address _controller, address _operator) public view returns
    (bool) {
-   return operators[_controller] == _operator;
+   return operators[_controller][_operator];
  }

```


3S-InfiniFi-M05

LPTVault::_redeem incorrectly assumes **EnumerableSet** maintains chronological ordering

Id	3S-InfiniFi-M05
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #4f02d39 .

Description

The **LPTVault** contract implements an ERC-7540 vault that allows users to deposit assets and request redemptions through an unwinding mechanism. When users call **_requestRedeem**, their unwinding position is tracked by storing **block.timestamp** in an **EnumerableSet.UintSet** mapped to their address. The **_redeem** function is responsible for processing claimable redemptions by retrieving the position at index **0**, verifying it is withdrawable, and executing the redemption.

The implementation assumes that the position at index **0** is always the oldest (and therefore most likely to be withdrawable first). However, OpenZeppelin's **EnumerableSet** does not maintain insertion order. When an element is removed via **remove()**, the internal implementation swaps the last element into the removed position to maintain a compact array. This breaks any assumed ordering.

Impact: After the first redemption, the user must wait for the last position to become withdrawable, and all positions in between will be stuck until that happens. This can result in users being unable to access their funds for extended periods even when those positions have completed the unwinding period.

Recommendation

Modify **_redeem** to iterate through the set and find the oldest (minimum timestamp) position instead of assuming index **0** contains it:

```
function _redeem(uint256, address _controller, address _receiver) internal
override returns (uint256 assetsOut) {
    EnumerableSet.UintSet storage _unwindings = unwindings[_controller];
```

```

-   require(_unwindings.length() > 0, NoPositions());
+   uint256 unwindingsLength = _unwindings.length();
+   require(unwindingsLength > 0, NoPositions());
-   uint256 unwindingTimestamp = _unwindings.at(0);
+   uint256 unwindingTimestamp = _unwindings.at(0);
+   for (uint256 i = 1; i < unwindingsLength; i++) {
+       uint256 timestamp = _unwindings.at(i);
+       if (timestamp < unwindingTimestamp) {
+           unwindingTimestamp = timestamp;
+       }
+   }
+
+   (uint256 receiptTokens, bool withdrawable) =
gateway.unwindingPosition(address(this), unwindingTimestamp);
    require(withdrawable, RedemptionNotClaimable(unwindingTimestamp));
    _unwindings.remove(0);
    gateway.withdraw(unwindingTimestamp);
    uint256 minAssetsOut = gateway.receiptToAsset(receiptTokens);
    IERC20(gateway.receiptToken()).forceApprove(address(gateway),
receiptTokens);
    return gateway.redeem(_receiver, receiptTokens, minAssetsOut);
}

```

3S-InfiniFi-M06

Slippage check in **MapleFarm::checkSlippage** modifier is ineffective due to incorrect comparison logic

Id	3S-InfiniFi-M06
Classification	Medium
Impact	Medium
Likelihood	High
Category	Bug
Status	Addressed in #db6b359 .

Description

The **MapleFarm** contract is a farm integration that deploys USDC into Maple's syrupUSDC pool and tracks pending redemptions. The **checkSlippage** modifier is intended to protect against excessive value loss during vault operations such as deposits and redemption requests by ensuring the total assets after an operation remain within an acceptable threshold relative to the assets before.

However, the current implementation calculates **minAssetsAfter** using **assetsAfter** instead of **assetsBefore**:

```
uint256 minAssetsAfter = assetsAfter.mulWadDown(maxSlippage);
require(assetsAfter >= minAssetsAfter, SlippageTooHigh(minAssetsAfter,
assetsAfter));
```

Since **maxSlippage** is set to **0.999999e18** (less than 1 WAD), the expression **assetsAfter.mulWadDown(maxSlippage)** will always produce a value smaller than **assetsAfter**. This means the condition **assetsAfter >= minAssetsAfter** will always evaluate to **true**, rendering the slippage protection completely ineffective.

Additionally, the **checkSlippage** modifier is applied to **MapleFarm::vaultRequestRedeem**, which only initiates a redemption request and escrows shares in the withdrawal manager. The actual asset transfer occurs later through the Maple withdrawal queue. Checking slippage on this function serves no purpose since the contract's asset balance does not change at the time of the request.

Recommendation

Update the **checkSlippage** modifier to calculate **minAssetsAfter** based on **assetsBefore**, and remove the modifier from **MapleFarm::vaultRequestRedeem** since slippage checking is not applicable to redemption requests:

```

modifier checkSlippage() {
    uint256 assetsBefore = assets();
    _;
    uint256 assetsAfter = assets();
-   uint256 minAssetsAfter = assetsAfter.mulWadDown(maxSlippage);
+   uint256 minAssetsAfter = assetsBefore.mulWadDown(maxSlippage);
    require(assetsAfter >= minAssetsAfter, SlippageTooHigh(minAssetsAfter,
assetsAfter));
}

function vaultRequestRedeem(uint256 _shares)
    external
    whenNotPaused
-   checkSlippage
    onlyCoreRole(CoreRoles.FARM_SWAP_CALLER)
    returns (uint256)
{
    return IMaplePoolLike(syrupUsdc).requestRedeem(_shares, address(this));
}

```

3S-InfiniFi-L01

Incorrect locked-to-receipt conversion logic leads to broken asset conversion and potential withdrawal reverts

Id	3S-InfiniFi-L01
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #0e512aa .

Description

The **LPTVault** contract uses **receiptToLocked** and **lockedToReceipt** as internal helpers to convert between receipt tokens and underlying locked **LPT**, which are then used by **LPTVault::convertToAssets** and ultimately by **LPTVault::_withdraw**. The protocol defines **rateLPT** as the ratio of receipt tokens to **LPT** (scaled by 1e18), and the current implementation of **receiptToLocked** is consistent with this definition. However, **lockedToReceipt** does not implement the mathematical inverse of **receiptToLocked**; instead it applies an additional **.mulWadDown(receiptTokenPrice(_gateway))** factor, which depends on the price of the receipt token. This extra multiplication distorts the conversion result, causing **LPTVault::convertToAssets** to return incorrect asset amounts and making any withdrawal logic that relies on this conversion (such as **LPTVault::_withdraw**) prone to reverts or miscalculations when it is eventually used. Since the **_withdraw** function is currently unreachable, the impact of this issue is limited.

Recommendation

Adjust **LPTVault::lockedToReceipt** so that it is strictly the inverse of **LPTVault::receiptToLocked** based solely on **rateLPT**, and remove the extraneous dependency on **receiptTokenPrice(_gateway)** from the conversion formula.

3S-InfiniFi-L02

LPTVault::claimableRedeemRequest returns incorrect claimable shares due to incomplete iteration and unordered set assumptions

Id	3S-InfiniFi-L02
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #4f02d39 .

Description

The **LPTVault::claimableRedeemRequest** function is responsible for returning the total amount of shares that a controller can currently claim from their completed unwinding positions. The current implementation only inspects the element at index 0 of the **unwindings** set and returns zero if that particular position is not yet withdrawable. This logic is flawed for two reasons:

1. **Multiple withdrawable positions are ignored:** When the first position is withdrawable, the function returns only that position's locked shares. However, other positions in the set may also have completed their unwinding period and be withdrawable. These are not accounted for.
2. **Unordered set behavior:** The **unwindings** mapping uses OpenZeppelin's **EnumerableSet.UintSet**, which does not maintain insertion order. When elements are removed, the set reorders by swapping the last element into the removed position. As a result, the element at index 0 is not guaranteed to be the oldest unwinding position. If a newer position happens to be at index 0 and is not yet withdrawable, the function returns zero even when older positions in the set are ready to be claimed.

This causes **claimableRedeemRequest** to underreport or incorrectly report zero claimable shares.

Recommendation

Iterate over all positions in the **unwindings** set and accumulate the shares from all withdrawable positions:

```

function claimableRedeemRequest(uint256, address _controller) public view
override returns (uint256 shares) {
    EnumerableSet.UintSet storage _unwindings = unwindings[_controller];
-   if (_unwindings.length() == 0) return 0;
+   uint256 unwindingsLength = _unwindings.length();
+   if (unwindingsLength == 0) return 0;
-   (uint256 receiptTokens, bool withdrawable) =
gateway.unwindingPosition(address(this), _unwindings.at(0));
-   return withdrawable ? gateway.receiptToLocked(receiptTokens,
unwindingEpochs) : 0;
+   for (uint256 i = 0; i < unwindingsLength; i++) {
+       (uint256 receiptTokens, bool withdrawable) =
gateway.unwindingPosition(address(this), _unwindings.at(i));
+       if (withdrawable) {
+           shares += gateway.receiptToLocked(receiptTokens, unwindingEpochs);
+       }
+   }
}

```

3S-InfiniFi-L03

LPTVault::pendingRedeemRequest returns incorrect value due to incorrect assumption about **EnumerableSet** ordering

Id	3S-InfiniFi-L03
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #4f02d39 .

Description

The **pendingRedeemRequest** function is responsible for calculating the total shares that are currently in the unwinding process but are not yet claimable for a given controller. This value is essential for users and integrating contracts to accurately track pending redemption requests within the ERC-7540 vault implementation.

The function iterates over the **unwindings** set and sums the shares of positions that are not yet withdrawable. However, the logic incorrectly assumes that the oldest unwinding position (the one closest to becoming withdrawable) will always reside at index 0 of the **EnumerableSet**. Based on this assumption, the function skips index 0 when the position at that index is withdrawable, treating it as already "claimable" rather than "pending."

This assumption is flawed because **EnumerableSet** from OpenZeppelin does not preserve insertion order after elements are removed. When **remove()** is called, the set uses a "swap and pop" algorithm, swapping the removed element with the last element before deletion, which disrupts the original ordering. As a result, after any removal operation, the position at index 0 may not be the oldest unwinding position.

Recommendation

Remove the index-based assumption and instead skip all withdrawable positions regardless of their index in the set:

```
function pendingRedeemRequest(uint256, address _controller) public view override
returns (uint256 shares) {
    EnumerableSet.UintSet storage _unwindings = unwindings[_controller];
    uint256 unwindingsLength = _unwindings.length();
```



```

if (unwindingsLength == 0) return 0;
for (uint256 i = 0; i < unwindingsLength; i++) {
    (uint256 receiptTokens, bool withdrawable) =
gateway.unwindingPosition(address(this), _unwindings.at(i));
-    // skip first item if it is withdrawable as it is now in `claimable` state
-    if (withdrawable && i == 0) continue;
+    if (withdrawable) continue;
    shares += gateway.receiptToLocked(receiptTokens, unwindingEpochs);
}
}

```

3S-InfiniFi-L04

ERC7540::supportsInterface uses incorrect parameter type, breaking ERC-165 compliance

Id	3S-InfiniFi-L04
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #4f02d39 .

Description

The **ERC7540** contract is an abstract implementation of the ERC-7540 asynchronous vault standard with ERC-7575 extensions. The **supportsInterface** function is intended to allow external contracts and applications to query whether the contract supports a specific interface, following the ERC-165 introspection standard. This is essential for protocol interoperability, as integrators rely on this function to determine supported capabilities before interacting with the vault.

However, the current implementation declares the **_interfaceID** parameter as **bytes32** instead of the ERC-165 mandated **bytes4** type. This deviation causes the function signature to differ from the standard **supportsInterface(bytes4)** selector, meaning:

1. External contracts calling the standard ERC-165 **supportsInterface(bytes4)** function will not hit this implementation
2. The contract will not be recognized as ERC-165 compliant by any standard tooling or integrating protocols
3. ERC-7540 interface detection will not work as intended

Recommendation

Change the parameter type from **bytes32** to **bytes4** to comply with ERC-165:

```
- function supportsInterface(bytes32 _interfaceID) external pure returns (bool) {
+ function supportsInterface(bytes4 _interfaceID) external pure returns (bool) {
    return _interfaceID == ERC7540_7575_ID;
```

}

3S-InfiniFi-L05

ERC7540::supportsInterface returns incorrect interface ID for single-asset vault

Id	3S-InfiniFi-L05
Classification	Low
Impact	Low
Likelihood	High
Category	Bug
Status	Addressed in #25b834c .

Description

The **ERC7540::supportsInterface** function is used to indicate which ERC interfaces the vault contract supports, enabling external integrators and protocols to programmatically detect the vault's capabilities. This is essential for proper integration with ERC-7540 compliant systems.

According to the EIP-7540 specification, vaults that implement a single underlying **asset** (as opposed to multi-asset vaults defined in ERC-7575) must return **true** for the interface ID **0xe3bc4e65**. The current implementation incorrectly returns **true** for **0x2f0a18c5**, which is the interface ID for ERC-7575 multi-asset vaults.

The **ERC7540** contract implements a single-asset vault pattern, as evidenced by the singular **asset** immutable variable and the absence of any multi-asset logic. Therefore, the contract should advertise support for the single-asset interface.

This causes integrators relying on interface detection to misidentify the vault type, potentially leading to integration issues or incorrect assumptions about the vault's capabilities.

Recommendation

Update the comparison to use the correct single-asset vault interface ID as specified in EIP-7540:

```
- bytes4 public constant ERC7540_7575_ID = 0x2f0a18c5;  
  function supportsInterface(bytes32 _interfaceID) external pure returns (bool) {  
-    return _interfaceID == ERC7540_7575_ID;  
+    return _interfaceID == 0xe3bc4e65;  
  }
```

3S-InfiniFi-N01

Missing reuse of **receiptTokenPrice** return value in **GatewayLib.lockedToReceipt** leads to redundant external call

Id	3S-InfiniFi-N01
Classification	None
Category	Suggestion
Status	Addressed in #242ea18 .

Description

The **GatewayLib::lockedToReceipt** function converts a locked token amount into its corresponding receipt token amount. In its current implementation, **lockedToReceipt** calls **receiptTokenPrice(_gateway)** and then immediately issues a second call instead of using the initially returned value, leaving the first result unused.

Recommendation

Remove the redundant call **receiptTokenPrice(_gateway)**.

3S-InfiniFi-N02

Missing zero-input handling in STVault conversion functions leads to inconsistent behavior and unnecessary attack surface

Id	3S-InfiniFi-N02
Classification	None
Category	Suggestion
Status	Addressed in #5587118 .

Description

The **STVault::convertToShares** and **STVault::convertToAssets** functions are helper view functions intended to map between asset amounts and share amounts for integrators and off-chain tooling, similar to standard ERC-4626 behavior. In **LPTVault**, these functions explicitly return **0** when the input **assets** or **shares** is **0**, while **STVault** omits this check and applies the full conversion logic even for zero input. This inconsistency can cause different behavior across otherwise similar vaults, and it slightly enlarges the vault's attack surface by preserving an unnecessary edge case in conversion logic.

Recommendation

Align **STVault::convertToShares** and **STVault::convertToAssets** with **LPTVault** by short-circuiting zero inputs to return **0**, thereby standardizing behavior and removing the redundant edge case from the conversion logic.

3S-InfiniFi-N03

Missing zero-address validation in AaveV3Farm and MapleFarm constructors

Id	3S-InfiniFi-N03
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **AaveV3Farm** and **MapleFarm** constructors accept critical address parameters that are stored as immutable variables but do not validate these inputs against **address(0)**. If any of these parameters are incorrectly set to the zero address during deployment, the contracts become permanently non-functional with no way to fix them since immutable variables cannot be modified after construction.

In **AaveV3Farm**, the constructor accepts **_aToken** and **_aaveV3Pool** without validation:

```
constructor(address _aToken, address _aaveV3Pool, address _core, address
_assetToken) Farm(_core, _assetToken) {
    aToken = _aToken;
    lendingPool = _aaveV3Pool;
}
```

In **MapleFarm**, the constructor accepts **_usdc**, **_syrupUsdc**, and **_syrupRouter** without validation:

```
constructor(address _core, address _accounting, address _usdc, address
_syrupUsdc, address _syrupRouter)
MultiAssetFarmV2(_core, _usdc, _accounting)
{
    usdc = _usdc;
    syrupUsdc = _syrupUsdc;
    syrupRouter = _syrupRouter;
    _enableAsset(_usdc);
    _enableAsset(_syrupUsdc);
    maxSlippage = 0.999999e18;
```

```
}
```

If any of these addresses are set to zero, core operations such as deposits, withdrawals, and asset tracking will fail. Since **aToken**, **usdc**, **syrupUsdc**, and **syrupRouter** are immutable, the only remedy is to abandon the faulty deployment and redeploy with correct parameters.

Recommendation

Add zero-address validation for all critical address parameters in both constructors.